

```

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# ML Libraries
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
import xgboost as xgb
from sklearn.metrics import accuracy_score, f1_score,
classification_report, confusion_matrix

import warnings
warnings.filterwarnings('ignore')

# Continue with your analysis...

```

Reading the Data :

```

# Directly specify the path to the CSV file
file_path = r'C:\Users\PC\Documents\UPWORK\AYUSH\WineQT.csv'
df = pd.read_csv(file_path)
df.head

```

	citric acid	residual sugar	fixed acidity	volatile acidity
0	0.076	7.4	0.700	1.9
1	0.098	7.8	0.880	2.6
2	0.092	7.8	0.760	2.3
3	0.075	11.2	0.280	1.9
4	0.076	7.4	0.700	1.9
...	...	...	...	...
1138	0.076	6.3	0.510	2.3
1139	0.068	6.8	0.620	1.9

1140	6.2	0.600	0.08	2.0
0.090				
1141	5.9	0.550	0.10	2.2
0.062				
1142	5.9	0.645	0.12	2.0
0.075				

	free sulfur dioxide	total sulfur dioxide	density	pH
0	11.0	34.0	0.99780	3.51
0.56				
1	25.0	67.0	0.99680	3.20
0.68				
2	15.0	54.0	0.99700	3.26
0.65				
3	17.0	60.0	0.99800	3.16
0.58				
4	11.0	34.0	0.99780	3.51
0.56				
...	...	...	...	...
...				
1138	29.0	40.0	0.99574	3.42
0.75				
1139	28.0	38.0	0.99651	3.42
0.82				
1140	32.0	44.0	0.99490	3.45
0.58				
1141	39.0	51.0	0.99512	3.52
0.76				
1142	32.0	44.0	0.99547	3.57
0.71				

	alcohol	quality	Id
0	9.4	5	0
1	9.8	5	1
2	9.8	5	2
3	9.8	6	3
4	9.4	5	4
...	...	...	...
1138	11.0	6	1592
1139	9.5	6	1593
1140	10.5	5	1594
1141	11.2	6	1595
1142	10.2	5	1597

[1143 rows x 13 columns]>

```
del df['Id']
```

df

	fixed acidity	volatile acidity	citric acid	residual sugar
chlorides \				
0	7.4	0.700	0.00	1.9
0.076				
1	7.8	0.880	0.00	2.6
0.098				
2	7.8	0.760	0.04	2.3
0.092				
3	11.2	0.280	0.56	1.9
0.075				
4	7.4	0.700	0.00	1.9
0.076				
...	...	...	...	...
...				
1138	6.3	0.510	0.13	2.3
0.076				
1139	6.8	0.620	0.08	1.9
0.068				
1140	6.2	0.600	0.08	2.0
0.090				
1141	5.9	0.550	0.10	2.2
0.062				
1142	5.9	0.645	0.12	2.0
0.075				
	free sulfur dioxide	total sulfur dioxide	density	pH
sulphates \				
0	11.0	34.0	0.99780	3.51
0.56				
1	25.0	67.0	0.99680	3.20
0.68				
2	15.0	54.0	0.99700	3.26
0.65				
3	17.0	60.0	0.99800	3.16
0.58				
4	11.0	34.0	0.99780	3.51
0.56				
...	...	...	...	...
...				
1138	29.0	40.0	0.99574	3.42
0.75				
1139	28.0	38.0	0.99651	3.42
0.82				
1140	32.0	44.0	0.99490	3.45
0.58				
1141	39.0	51.0	0.99512	3.52
0.76				
1142	32.0	44.0	0.99547	3.57
0.71				

	alcohol	quality
0	9.4	5
1	9.8	5
2	9.8	5
3	9.8	6
4	9.4	5
...	...	...
1138	11.0	6
1139	9.5	6
1140	10.5	5
1141	11.2	6
1142	10.2	5

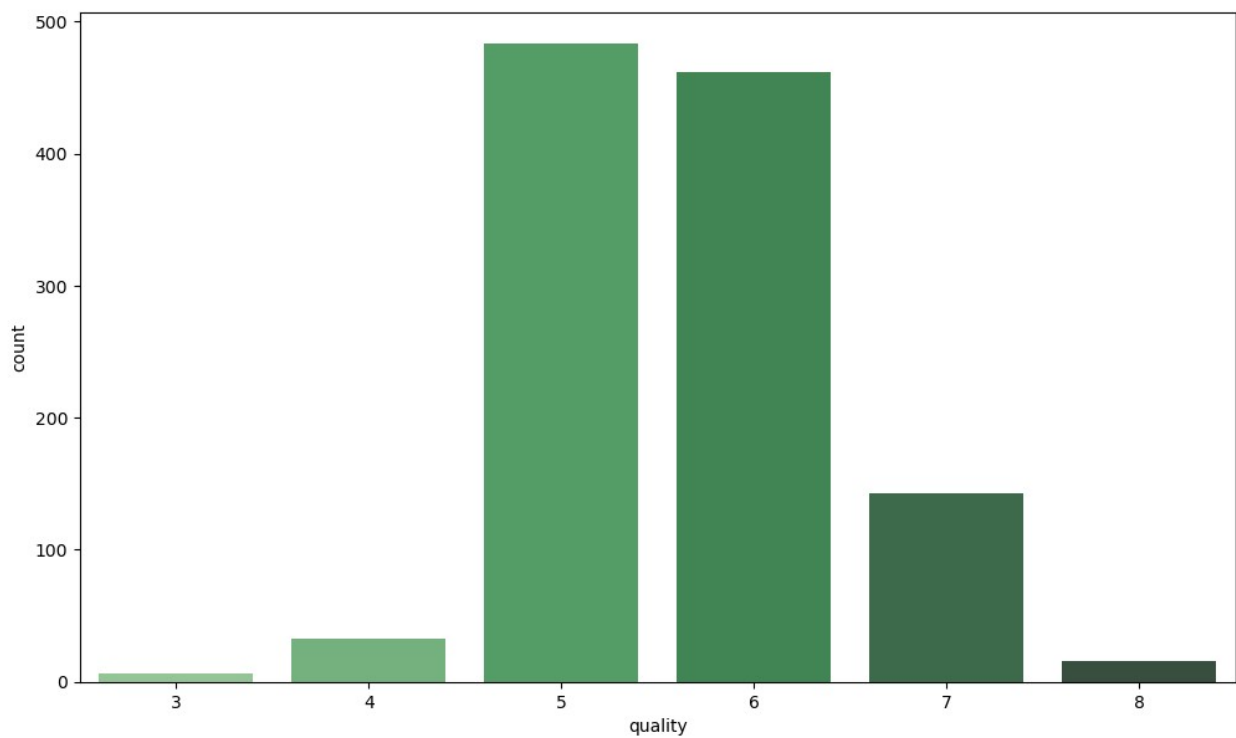
[1143 rows x 12 columns]

```
df.describe().T.style.background_gradient(cmap='Greens')
```

```
<pandas.io.formats.style.Styler at 0x1e1ec648490>
```

Exploratory analysis

```
plt.figure(figsize=(12,7))
sns.countplot(x='quality',data=df,palette='Greens_d')
plt.show()
```



Correlation of dataset is given below

```
df.corr()
```

	fixed acidity	volatile acidity	citric acid	\
fixed acidity	1.000000	-0.250728	0.673157	
volatile acidity	-0.250728	1.000000	-0.544187	
citric acid	0.673157	-0.544187	1.000000	
residual sugar	0.171831	-0.005751	0.175815	
chlorides	0.107889	0.056336	0.245312	
free sulfur dioxide	-0.164831	-0.001962	-0.057589	
total sulfur dioxide	-0.110628	0.077748	0.036871	
density	0.681501	0.016512	0.375243	
pH	-0.685163	0.221492	-0.546339	
sulphates	0.174592	-0.276079	0.331232	
alcohol	-0.075055	-0.203909	0.106250	
quality	0.121970	-0.407394	0.240821	

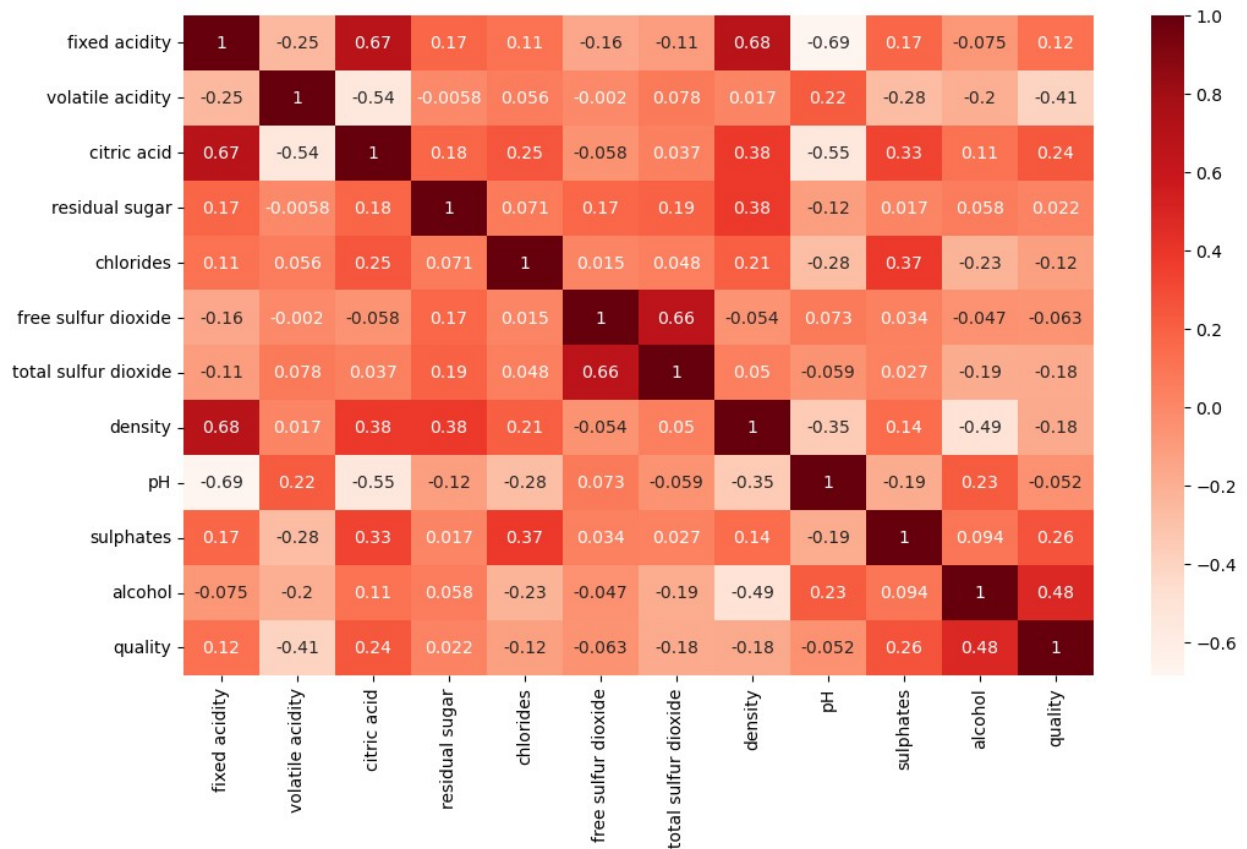
	residual sugar	chlorides	free sulfur
dioxide \			
fixed acidity	0.171831	0.107889	-0.164831
volatile acidity	-0.005751	0.056336	-0.001962
citric acid	0.175815	0.245312	-0.057589
residual sugar	1.000000	0.070863	0.165339
chlorides	0.070863	1.000000	0.015280
free sulfur dioxide	0.165339	0.015280	1.000000
total sulfur dioxide	0.190790	0.048163	0.661093
density	0.380147	0.208901	-0.054150
pH	-0.116959	-0.277759	0.072804
sulphates	0.017475	0.374784	0.034445
alcohol	0.058421	-0.229917	-0.047095
quality	0.022002	-0.124085	-0.063260

	total sulfur dioxide	density	pH
sulphates \			
fixed acidity	-0.110628	0.681501	-0.685163
0.174592			
volatile acidity	0.077748	0.016512	0.221492
0.276079			-

citric acid	0.036871	0.375243	-0.546339
0.331232			
residual sugar	0.190790	0.380147	-0.116959
0.017475			
chlorides	0.048163	0.208901	-0.277759
0.374784			
free sulfur dioxide	0.661093	-0.054150	0.072804
0.034445			
total sulfur dioxide	1.000000	0.050175	-0.059126
0.026894			
density	0.050175	1.000000	-0.352775
0.143139			
pH	-0.059126	-0.352775	1.000000
0.185499			
sulphates	0.026894	0.143139	-0.185499
1.000000			
alcohol	-0.188165	-0.494727	0.225322
0.094421			
quality	-0.183339	-0.175208	-0.052453
0.257710			

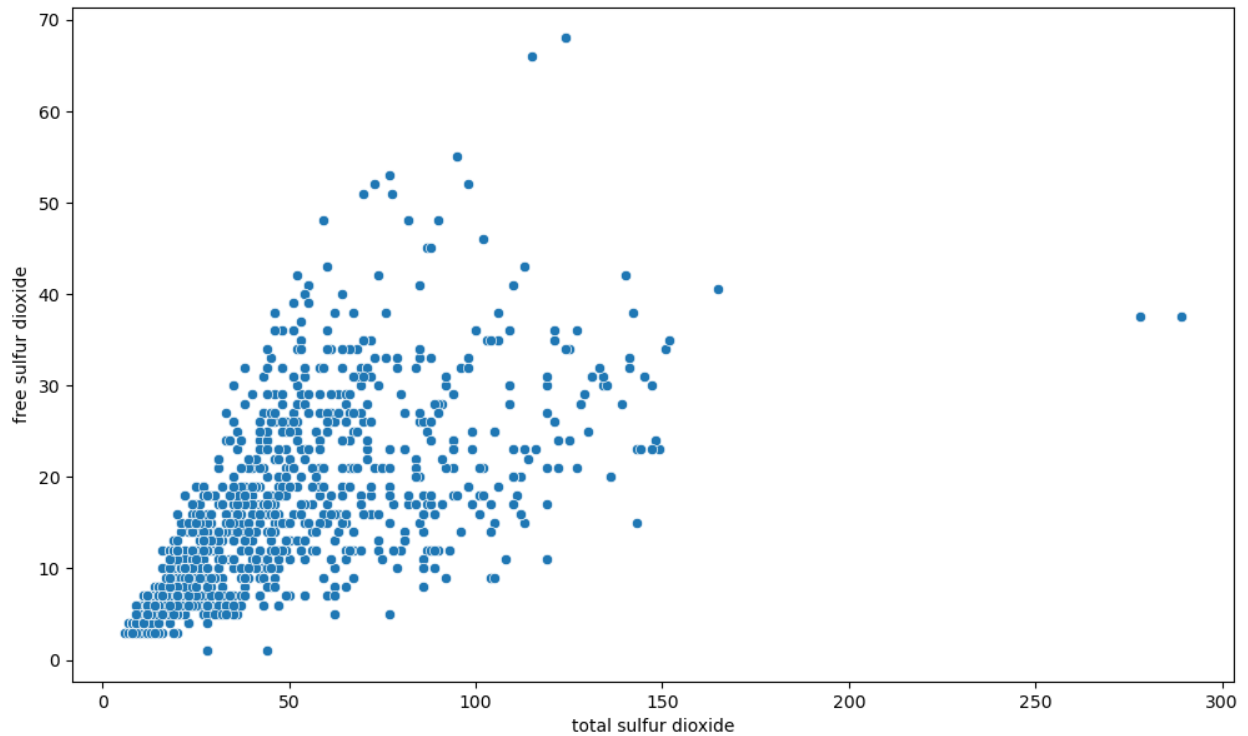
	alcohol	quality
fixed acidity	-0.075055	0.121970
volatile acidity	-0.203909	-0.407394
citric acid	0.106250	0.240821
residual sugar	0.058421	0.022002
chlorides	-0.229917	-0.124085
free sulfur dioxide	-0.047095	-0.063260
total sulfur dioxide	-0.188165	-0.183339
density	-0.494727	-0.175208
pH	0.225322	-0.052453
sulphates	0.094421	0.257710
alcohol	1.000000	0.484866
quality	0.484866	1.000000

```
plt.figure(figsize=(12,7))
sns.heatmap(df.corr(),annot=True,cmap='Reds')
plt.show()
```



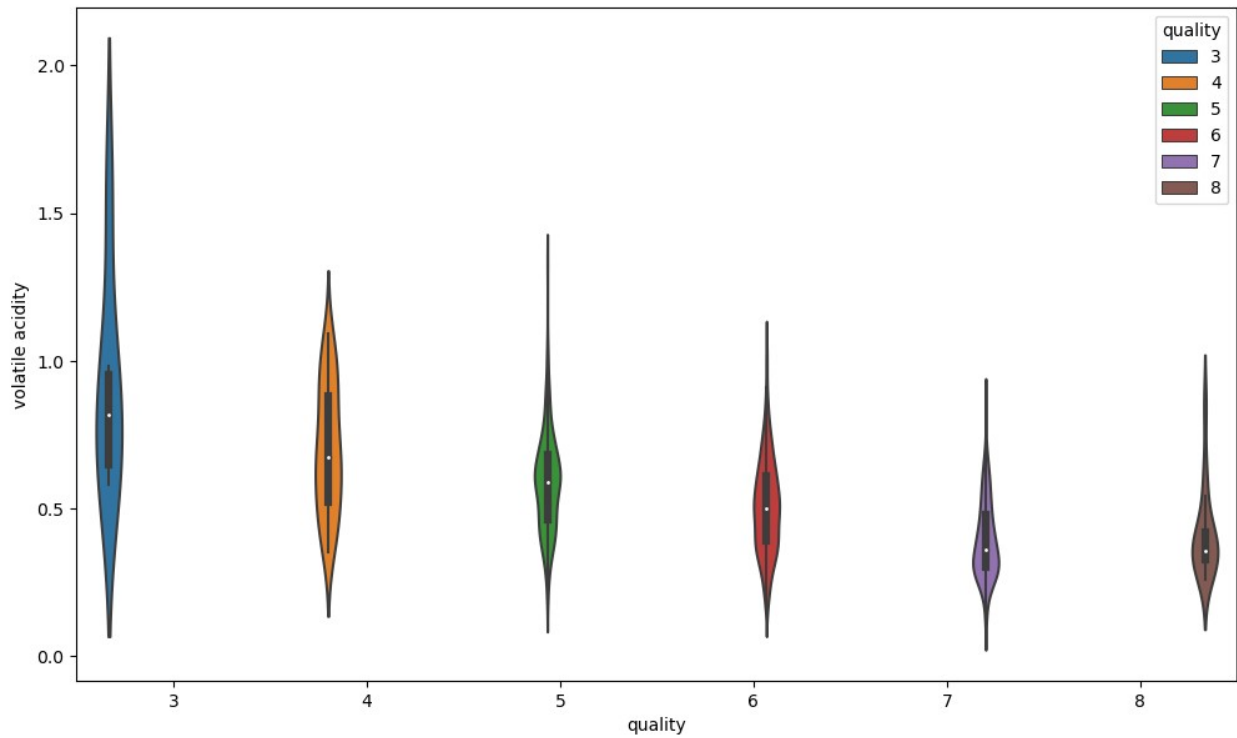
Plot b/w total sulphur dioxide and free sulphur dioxide

```
plt.figure(figsize=(12,7))
sns.scatterplot(x='total sulfur dioxide',y='free sulfur dioxide',data=df)
plt.show()
```

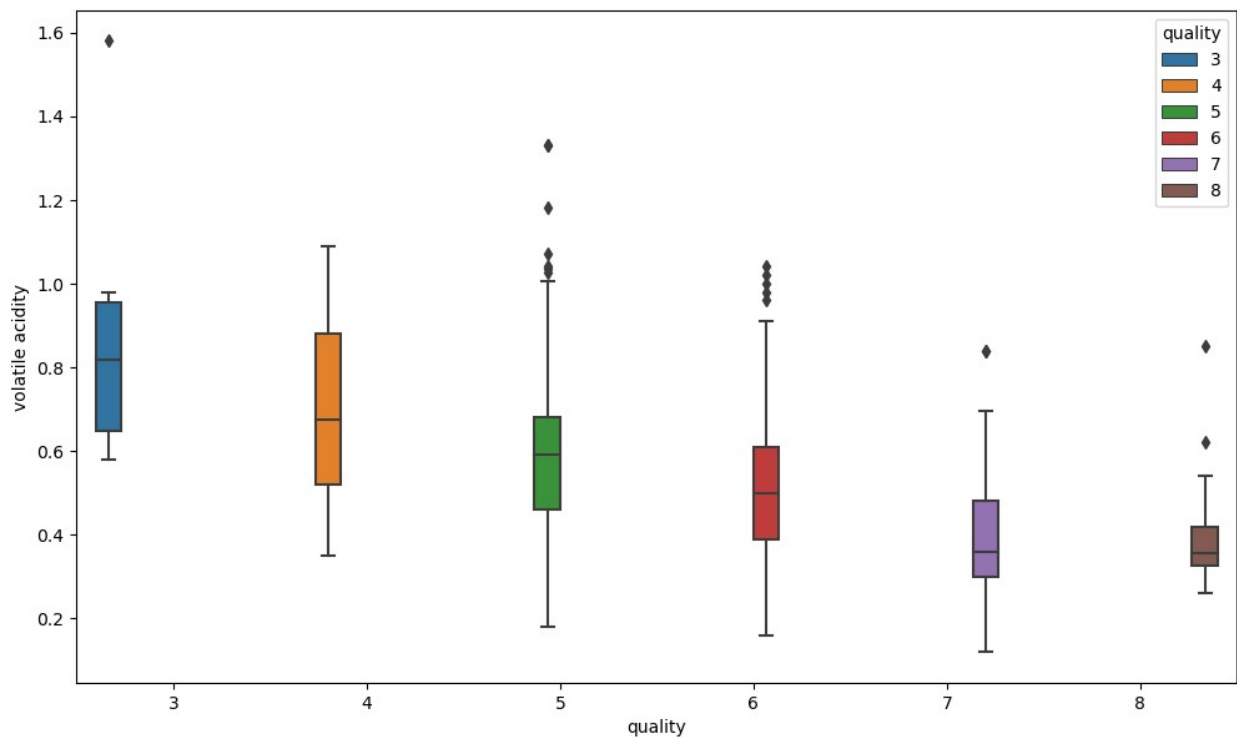


Quality of wine varies b/w 3-8, and the below plot give information about how the quality changes with respect to volatile acidity

```
plt.figure(figsize=(12,7))
sns.violinplot(x='quality',y='volatile acidity',data=df,hue='quality')
plt.show()
```



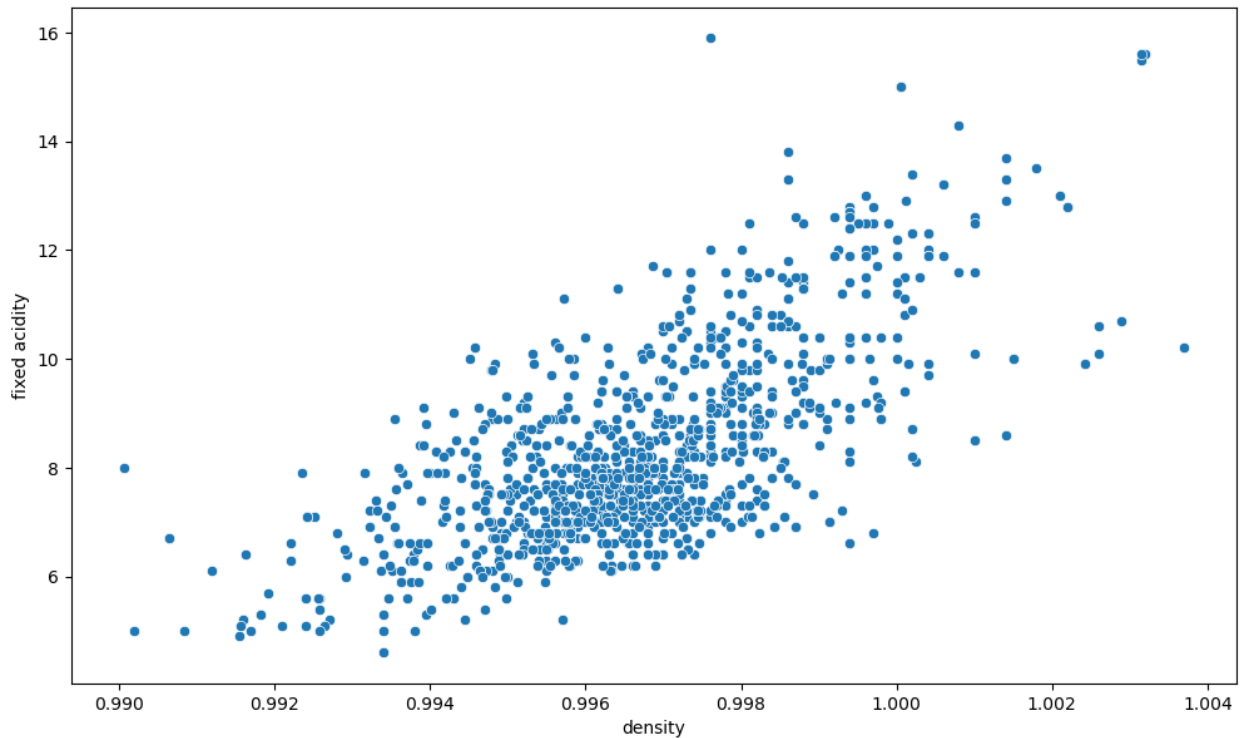
```
plt.figure(figsize=(12,7))
sns.boxplot(x='quality',y='volatile acidity',data=df,hue='quality')
plt.show()
```



The below plot explains in detail about how the density of wine varies with the fixed acidity

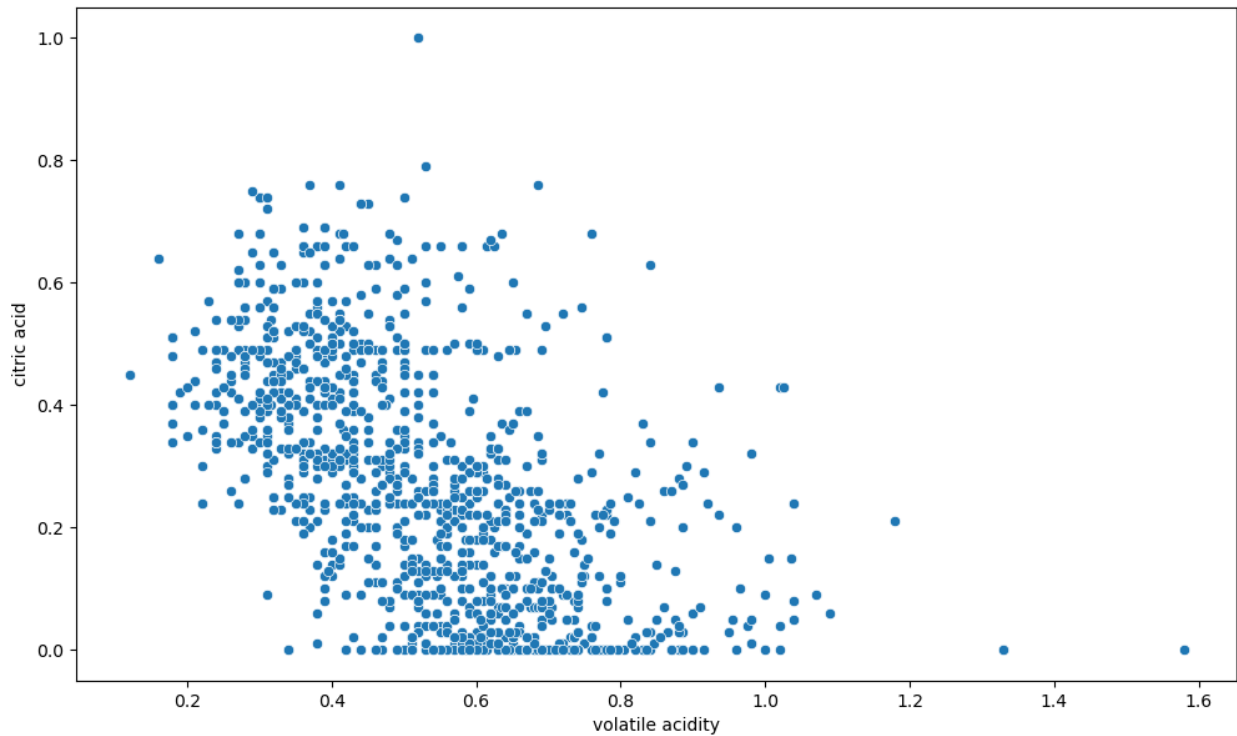
```
plt.figure(figsize=(12,7))  
sns.scatterplot(x='density',y='fixed acidity',data=df)
```

```
<Axes: xlabel='density', ylabel='fixed acidity'>
```



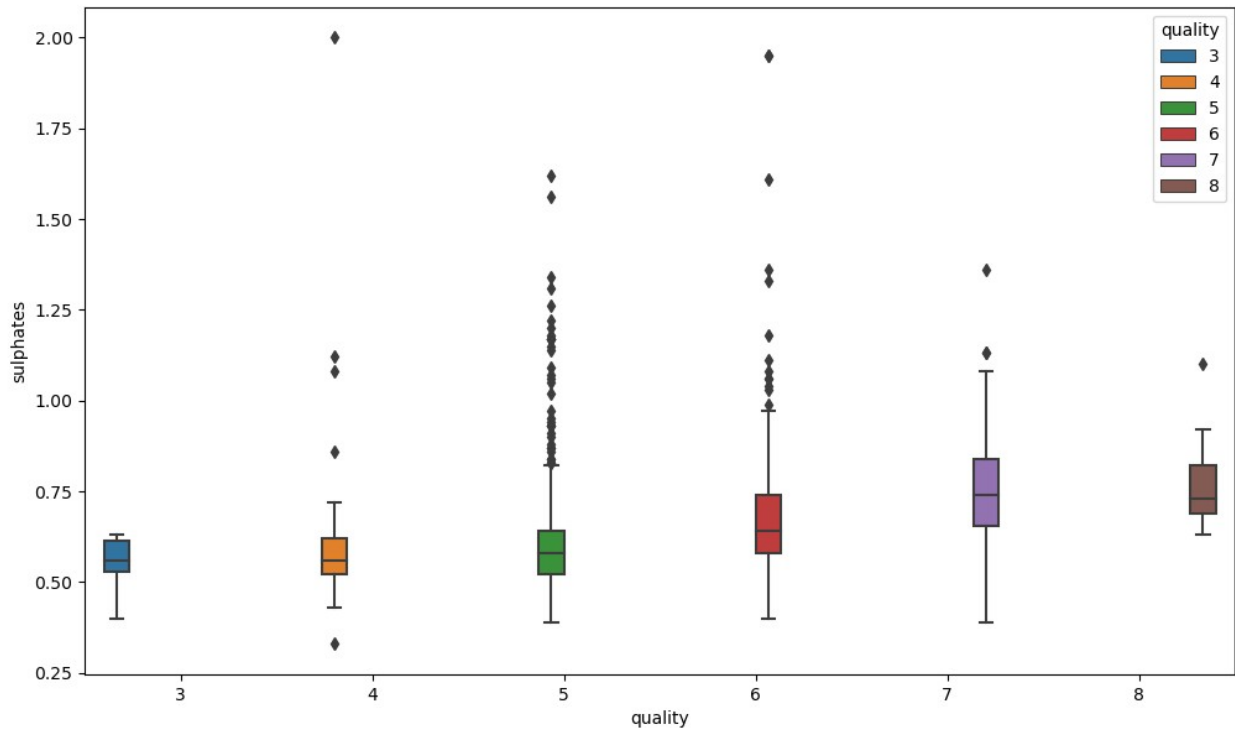
The below plot explains about how volatile acidity of wine changes with respect to citric acid

```
plt.figure(figsize=(12,7))  
sns.scatterplot(x='volatile acidity',y='citric acid',data=df)  
plt.show()
```



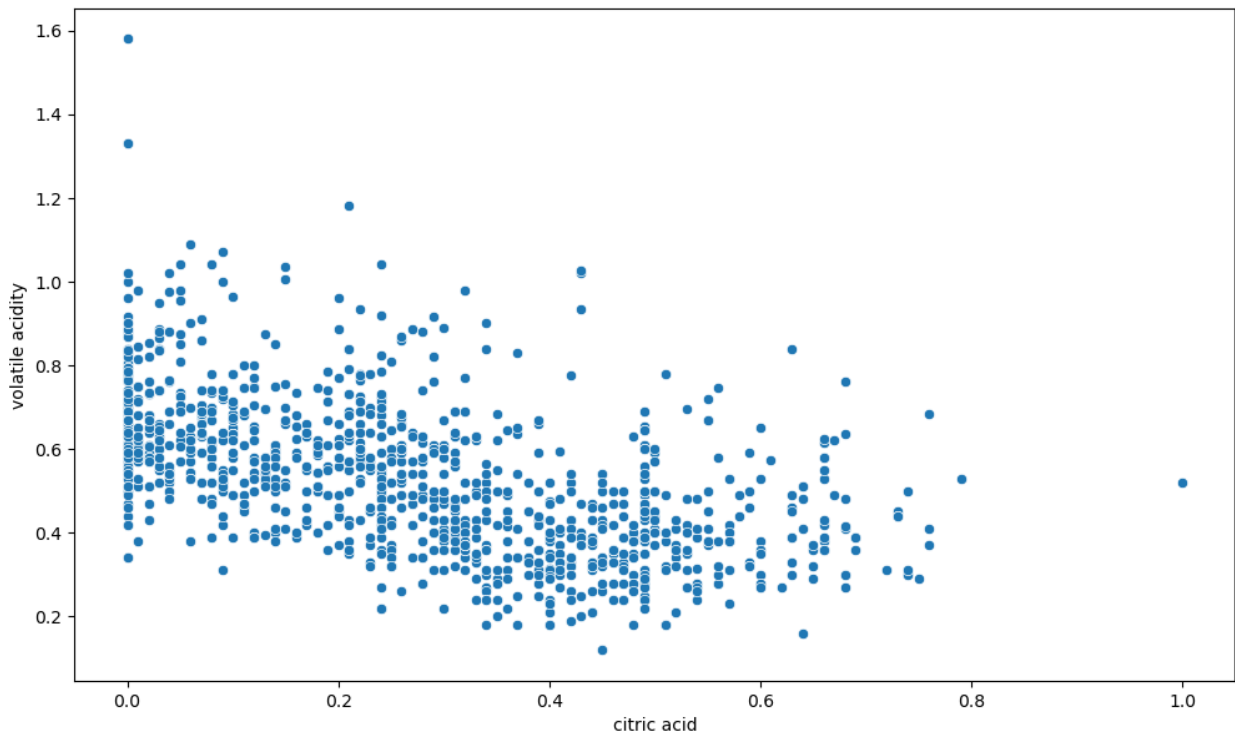
The below plot explains how quality of wine varies in according with sulphates in it

```
plt.figure(figsize=(12,7))  
sns.boxplot(x='quality',y='sulphates',data=df,hue='quality')  
plt.show()
```



The below plot describes about citric acid and volatile acidity

```
plt.figure(figsize=(12,7))
sns.scatterplot(x='citric acid',y='volatile acidity',data=df)
plt.show()
```



Data Preprocessing

```
for col in ['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'alcohol']:
    df[col]=df[col]/df[col].max()
```

Splitting the dataset as 20% for testing and 80% for training

```
feature=np.array(df[['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'alcohol']])
label=np.array(df['quality'])
```

```
xtrain,xtest,ytrain,ytest=train_test_split(feature,label,test_size=0.2,random_state=0)
```

Preparing for Models

```
model_comp={}
```

1.Logistic Regression

```
lgr=LogisticRegression()
lgr.fit(xtrain,ytrain)
y1=lgr.predict(xtest)
print(accuracy_score(ytest,y1))
print(f1_score(ytest,y1,average='weighted'))
print(classification_report(ytest,y1))
```

```
model_comp['logistic regression']=[accuracy_score(y1,ytest),f1_score(ytest,y1,average='weighted')]
```

0.6375545851528385

0.5818306114188381

	precision	recall	f1-score	support
3	0.00	0.00	0.00	1
4	0.00	0.00	0.00	7
5	0.69	0.82	0.75	100
6	0.58	0.70	0.63	92
7	0.00	0.00	0.00	27
8	0.00	0.00	0.00	2
accuracy			0.64	229
macro avg	0.21	0.25	0.23	229
weighted avg	0.54	0.64	0.58	229

2. Decision Tree

```
dt=DecisionTreeClassifier()
dt.fit(xtrain,ytrain)
y2=dt.predict(xtest)

print(accuracy_score(ytest,y2))
print(f1_score(ytest,y2,average='weighted'))
print(classification_report(ytest,y2))

model_comp['decision
tree']=[accuracy_score(y2,ytest),f1_score(ytest,y2,average='weighted')
]
```

0.5895196506550219
0.5924025998261806

	precision	recall	f1-score	support
3	0.00	0.00	0.00	1
4	0.09	0.14	0.11	7
5	0.67	0.67	0.67	100
6	0.59	0.58	0.58	92
7	0.52	0.48	0.50	27
8	0.33	0.50	0.40	2
accuracy			0.59	229
macro avg	0.37	0.40	0.38	229
weighted avg	0.60	0.59	0.59	229

3 RandomForest

```
rf=RandomForestClassifier()
rf.fit(xtrain,ytrain)
y3=rf.predict(xtest)

print(accuracy_score(ytest,y3))
print(f1_score(ytest,y3,average='weighted'))
print(classification_report(ytest,y3))

model_comp['Random
forest']=[accuracy_score(y3,ytest),f1_score(ytest,y3,average='weighted')
]
```

0.6681222707423581
0.6521506345396003

	precision	recall	f1-score	support
3	0.00	0.00	0.00	1
4	0.00	0.00	0.00	7

5	0.72	0.76	0.74	100
6	0.61	0.68	0.65	92
7	0.67	0.52	0.58	27
8	0.00	0.00	0.00	2
accuracy			0.67	229
macro avg	0.33	0.33	0.33	229
weighted avg	0.64	0.67	0.65	229

4 KNN

```
k=KNeighborsClassifier()
k.fit(xtrain,ytrain)
y4=k.predict(xtest)

print(accuracy_score(ytest,y4))
print(f1_score(ytest,y4,average='weighted'))
print(classification_report(ytest,y4))

model_comp['KNN']=[accuracy_score(y4,ytest),f1_score(ytest,y4,average=
'weighted')]

0.5589519650655022
0.5395476430410928
```

	precision	recall	f1-score	support
3	0.00	0.00	0.00	1
4	0.00	0.00	0.00	7
5	0.64	0.69	0.66	100
6	0.51	0.58	0.54	92
7	0.33	0.22	0.27	27
8	0.00	0.00	0.00	2
accuracy			0.56	229
macro avg	0.25	0.25	0.25	229
weighted avg	0.53	0.56	0.54	229

5 SVC

```
s=SVC()
s.fit(xtrain,ytrain)
y5=s.predict(xtest)

print(accuracy_score(ytest,y5))
print(f1_score(ytest,y5,average='weighted'))
print(classification_report(ytest,y5))
```

```
model_comp['svc']=[accuracy_score(y5,ytest),f1_score(ytest,y5,average='weighted')]
```

```
0.6637554585152838
```

```
0.6069291484469779
```

	precision	recall	f1-score	support
3	0.00	0.00	0.00	1
4	0.00	0.00	0.00	7
5	0.73	0.81	0.77	100
6	0.60	0.77	0.68	92
7	0.00	0.00	0.00	27
8	0.00	0.00	0.00	2
accuracy			0.66	229
macro avg	0.22	0.26	0.24	229
weighted avg	0.56	0.66	0.61	229

Model Comparison

accuracy and f1 scores of all applied models are given below

```
df1=pd.DataFrame.from_dict(model_comp).T
df1.columns=['Accuracy','F1_score']
df1=df1.sort_values('F1_score',ascending=True)
df1.style.background_gradient(cmap='Greens')
<pandas.io.formats.style.Styler at 0x1e1f1125390>
```